

Term Report

By: Peeranat Patarakittisopol 65011446
Tonkla Pokaew 65011610
Soe Moe Htet 65011693



Overview

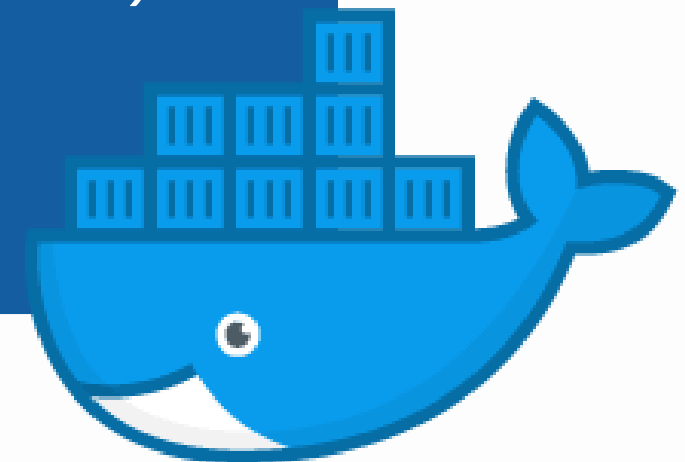
▶▶▶	Introduction	01
▶▶▶	Why Use Containers?	02
▶▶▶	Key Docker Concepts	03
▶▶▶	Frontend Service Setup	04
▶▶▶	Backend Service Setup	05
▶▶▶	Database Service Setup	06
▶▶▶	Networking & Communication	07
▶▶▶	Docker Compose	08





Introduction

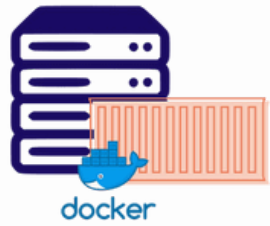
Docker is a platform for developing, shipping, and running applications inside containers. Containers are lightweight, standalone, and executable software packages that include everything the application needs to run (code, runtime, libraries, etc.).



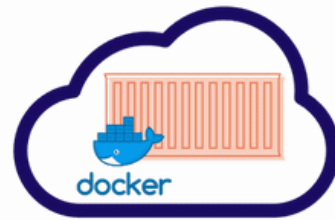
Benefits



” Works well on my laptop



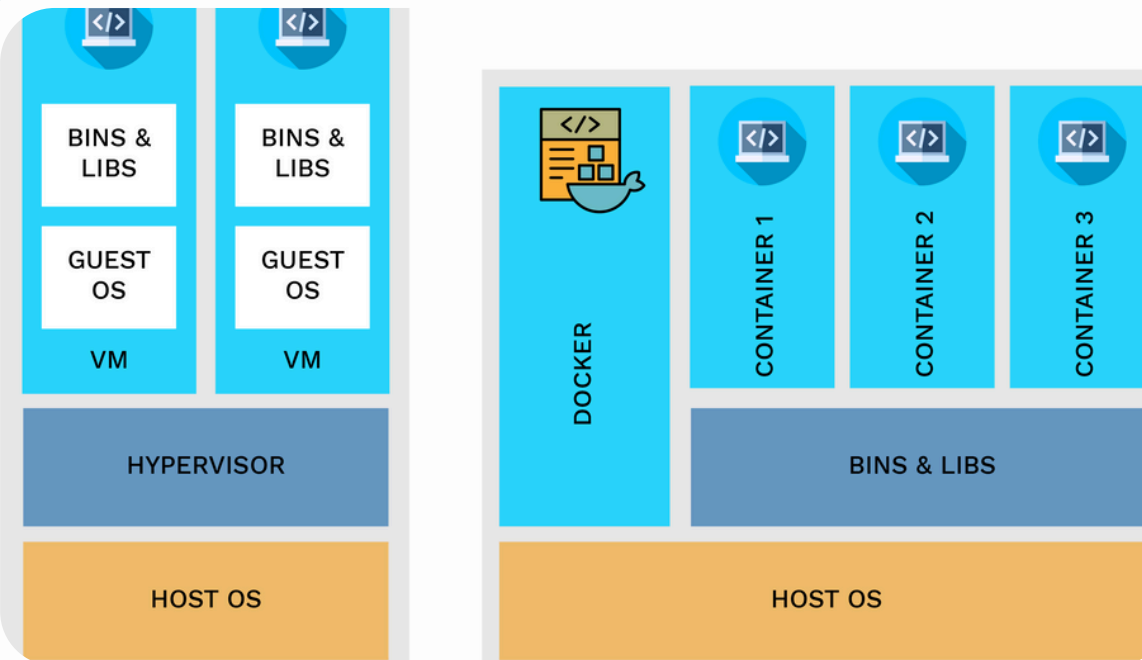
” All good also on server



” No issues on cloud!

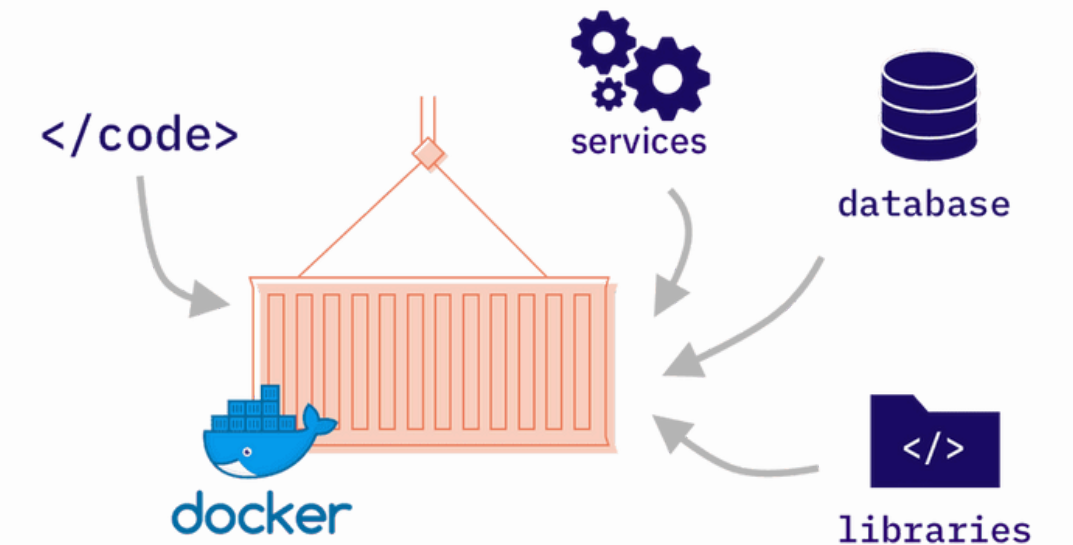
Consistency

Ensures that code runs the same regardless of where it's executed (e.g., locally, staging, production)



Resource Efficiency

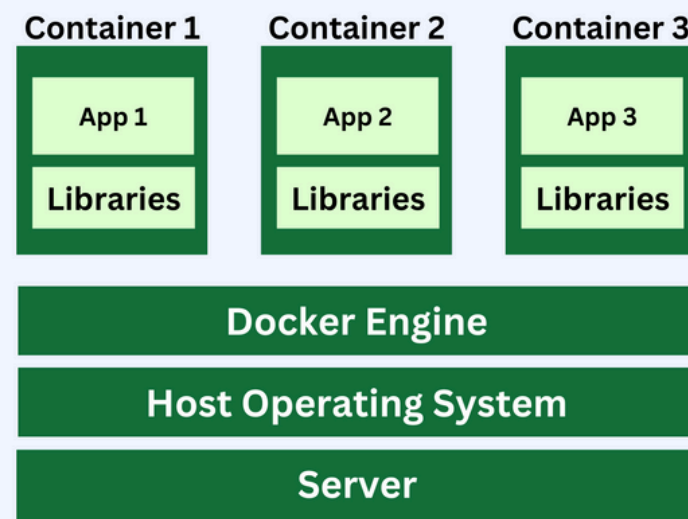
Containers share the host OS, reducing overhead and improving performance



Faster Deployment

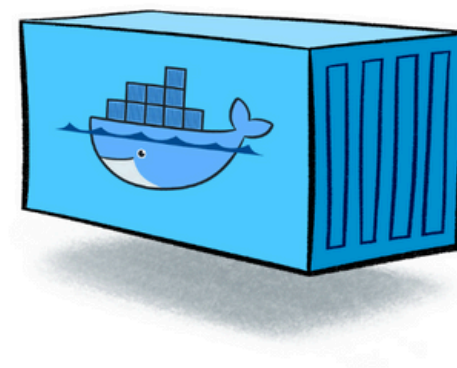
With all dependencies bundled, containers are lightweight and quick to start

Why Use Containers?



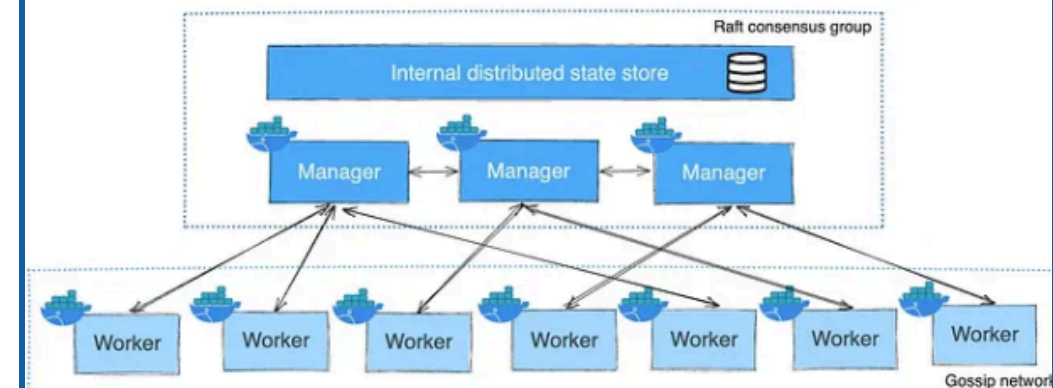
Isolation

Each container runs in its own isolated environment, ensuring that different applications or services don't interfere with each other.



Portability

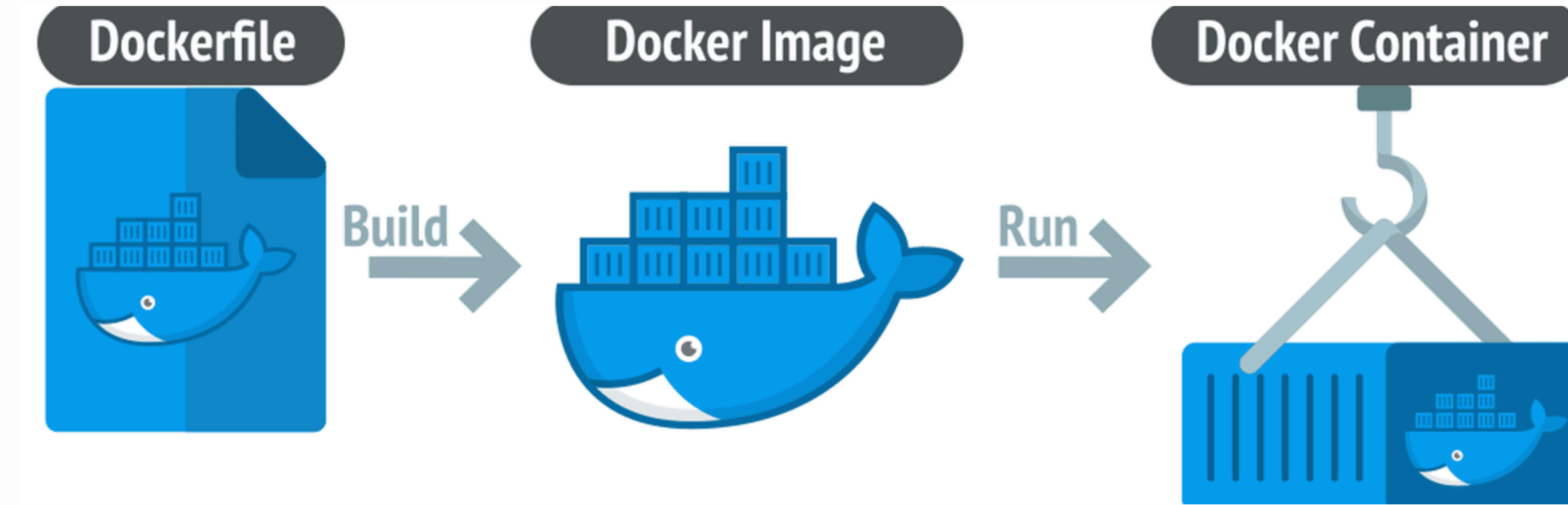
Containers can run on any system that supports Docker, regardless of underlying infrastructure.



Scalability

Containers are lightweight and can be quickly scaled up or down based on the demand, making them ideal for microservices and cloud-native architectures.

Key Docker Concepts



Dockerfile

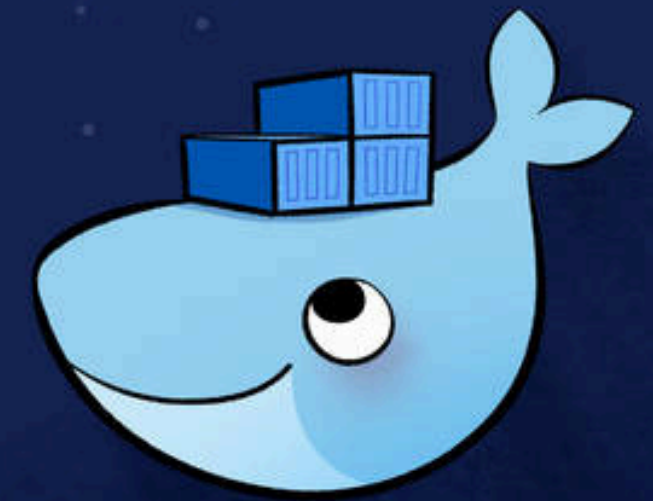
A file with instructions to build a Docker image.

Image

A blueprint containing the app code and dependencies.

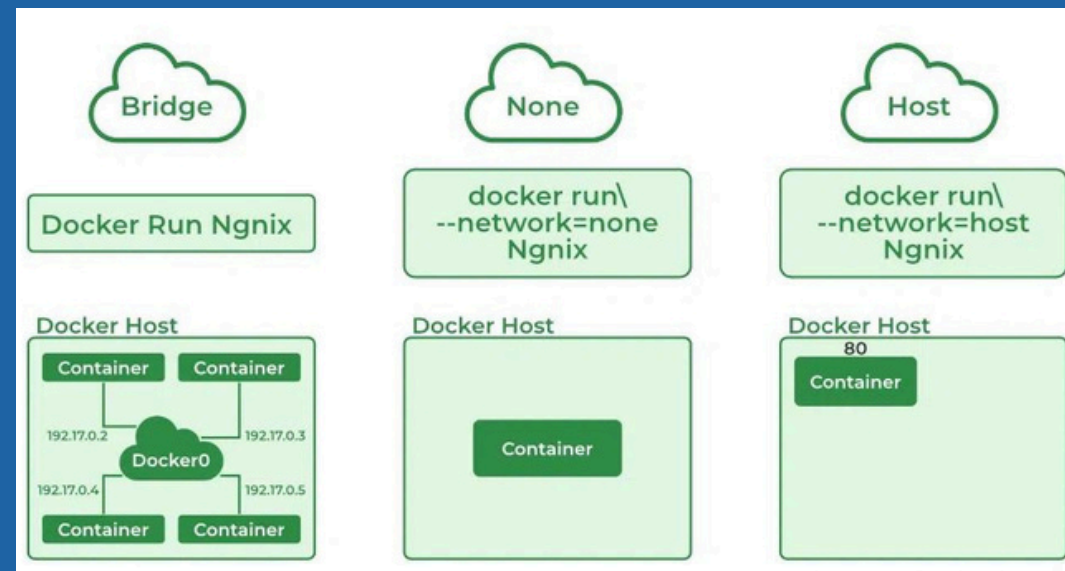
Container

A running instance of the image, lightweight and isolated.





Docker Network

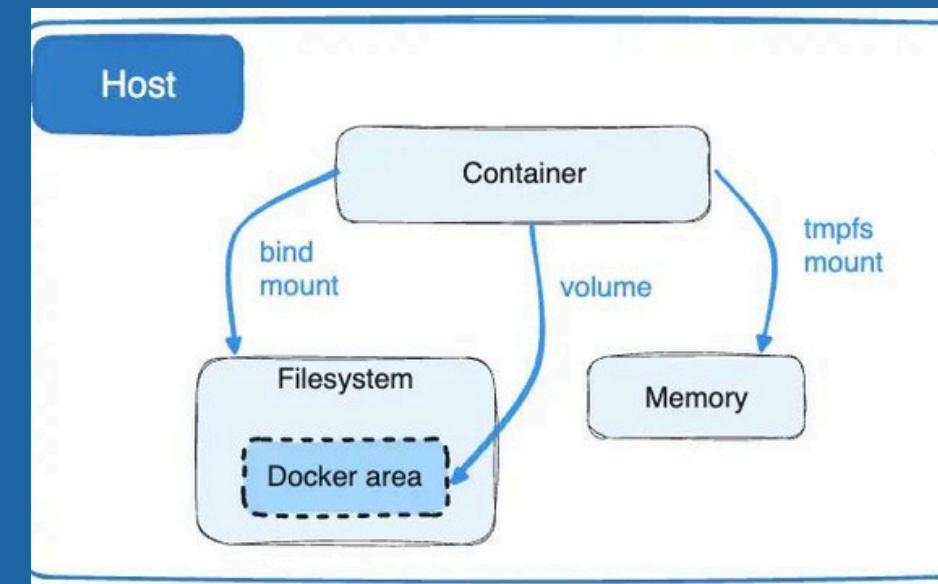


- Docker creates virtual networks to connect containers.
- Containers in the same network communicate using container names.

Example:

- A frontend can send requests to the backend using the backend's service name (e.g., backend:5000).

Docker Volume



- Volumes store persistent data used by containers.
- Data persists even when containers are removed or recreated.

Why It's Important:

- Keeps essential data (e.g., databases) safe and allows data sharing between containers.

The background is a solid dark blue. In the center, there is a white rectangular box with a thin white border. Inside this box, the text "FRONTEND SERVICE SETUP" is written in a bold, white, sans-serif font. The text is arranged in two lines: "FRONTEND" on the top line and "SERVICE SETUP" on the bottom line. Behind the white box, there are faint, light blue geometric shapes and lines, including a large stylized arrow pointing to the right, and some abstract shapes that resemble a web browser window and a document. The overall design is clean and modern, with a focus on the central text.

FRONTEND SERVICE SETUP

Tools Overview

React: A JavaScript library used for building dynamic and interactive user interfaces. It allows the creation of reusable components to manage the front-end efficiently.

Nginx: A high-performance web server used to serve static files like HTML, CSS, and JavaScript. It efficiently handles HTTP requests and can manage large numbers of concurrent connections, making it ideal for serving frontend applications.

HTML, CSS, JavaScript



Why Nginx?



Efficiency

Optimized for serving static content and handling many concurrent connections.



Lightweight

Uses fewer resources, making it ideal for simple frontend services and resource-constrained environments

Why Nginx?



Simplicity

Easy to configure for serving static files, allowing for quick deployment of web applications.



Load Balancing

Nginx has built-in load balancing capabilities, distributing incoming traffic across multiple servers to ensure high availability and improved performance.

Frontend Dockerfile

Stage 1: Build the React App

- React is built using Node.js.
- Dependencies are installed, and the app is built using Vite.
- Production files are output to the dist folder, ready for serving.

Stage 2: Serve the React App with Nginx

- Nginx (Alpine) serves the built files for performance.
- Production files are copied to Nginx's serving directory.
- A custom nginx.conf is added for configuration.

```
Dockerfile M X
frontend > Dockerfile > ...
1  # Stage 1: Build the React app
2  FROM node:18 AS build
3
4  WORKDIR /app
5
6  COPY package*.json ./
7
8  # Install dependencies
9  RUN npm install
10
11 # Copy the rest of the application code
12 COPY . .
13
14 # Build the application using Vite
15 RUN npm run build
16
17 # Stage 2: Serve the application with Nginx
18 FROM nginx:alpine
19
20 # Copy built files to the Nginx html directory
21 COPY --from=build /app/dist /usr/share/nginx/html
22
23 # Copy custom nginx configuration
24 COPY nginx.conf /etc/nginx/nginx.conf
25
26 # Expose port 80 to the outside world
27 EXPOSE 80
28
29 # Start Nginx when the container starts
30 CMD ["nginx", "-g", "daemon off;"]
31
```

Nginx Configuration

- **Server block:** It listens on port 80 and serves files from `/usr/share/nginx/html`, where the React build files are located.
- **Root directory:** The root directory is set to `/usr/share/nginx/html`, which is where the static assets are stored.
- **Caching:** The location `~* \.(js|css|png|jpg|jpeg|gif|svg|ico)$` block enables caching for static files, allowing the browser to cache files for up to 1 year (expires 1y). This improves performance by reducing the need to load static files repeatedly.

```
nginx.conf x
frontend > nginx.conf
1  user  nginx;
2  worker_processes  auto;
3  error_log  /var/log/nginx/error.log warn;
4  pid       /var/run/nginx.pid;
5
6  events {
7      worker_connections  1024;
8  }
9
10 http {
11     include        /etc/nginx/mime.types;
12     default_type    application/octet-stream;
13
14     log_format      main '$remote_addr - $remote_user [$time_local] "$request" '
15                          '$status $body_bytes_sent "$http_referer" '
16                          '"$http_user_agent" "$http_x_forwarded_for"';
17
18     access_log       /var/log/nginx/access.log  main;
19
20     sendfile         on;
21     keepalive_timeout 65;
22
23     # The server block goes inside the http block
24     server {
25         listen 80;
26         server_name localhost;
27
28         root /usr/share/nginx/html;
29
30         location / {
31             try_files $uri /index.html;
32         }
33
34         location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico)$ {
35             expires 1y;
36             add_header Cache-Control "public, max-age=31536000, immutable";
37         }
38     }
39 }
40
```

Running the Frontend Container

Once the Dockerfile is prepared, we build and run the frontend service with the following commands

Build the Docker Image

```
admin@admin Library-Manager % cd frontend
admin@admin frontend % docker build -t frontend .
```

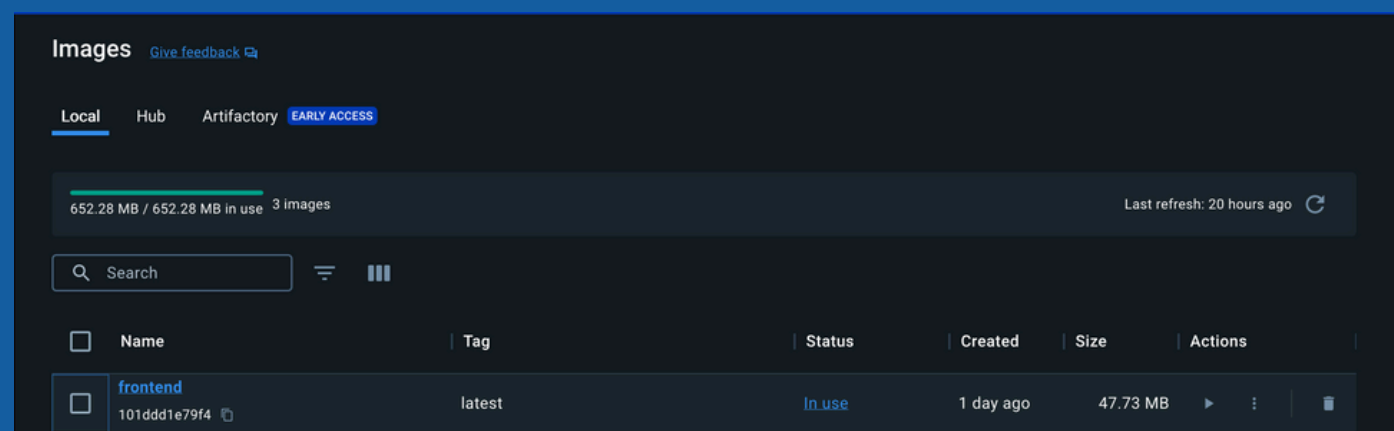
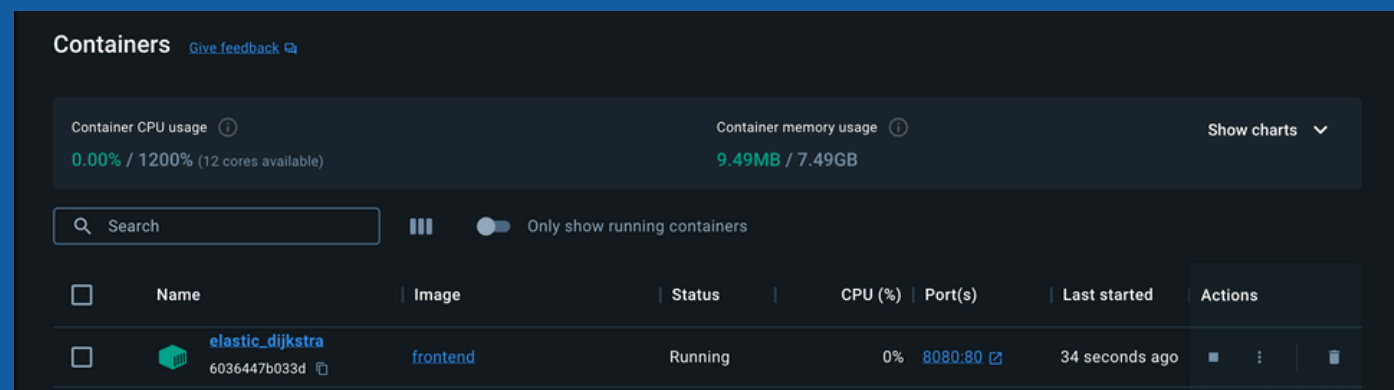
Run the Docker Container

```
frontend % docker run -d -p 8080:80 frontend
5a589328a24ec367e695f75e11cb17e01d00c435f54ed18
```

To stop the Container

```
docker stop <container_id>
```

```
admin@admin ~ % docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
1bb1f91cee95   frontend  "/docker-entrypoint..." 16 minutes ago Up 3 seconds  0.0.0.0:8080->80/tcp
admin@admin ~ % docker stop 1bb1f91cee95
admin@admin ~ % docker start 1bb1f91cee95
admin@admin ~ %
```





BACKEND SERVICE SETUP



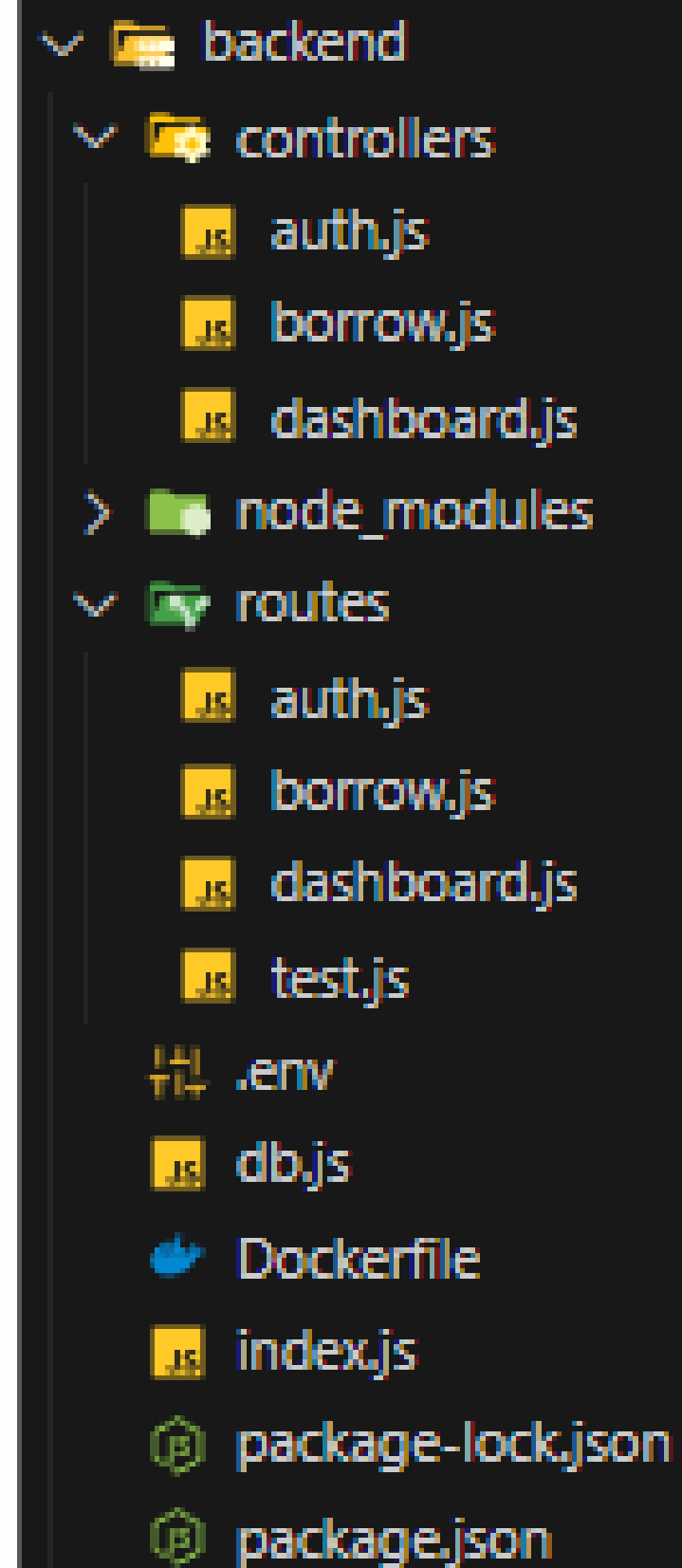
Tool Used: Express (Node.js)



```
backend >  index.js > ...
1  import express from 'express';
2  import cors from 'cors';
3  import dashboardRoute from './routes/dashboard.js';
4  import authRoute from './routes/auth.js';
5  import borrowRoute from './routes/borrow.js';
6  import testRoute from './routes/test.js';
7  import cookieParser from 'cookie-parser';
8
9  const app = express();
10
11  app.use(cors());
12  app.use(cookieParser());
13  app.use(express.json());
14
15
16  app.use('/dashboard', dashboardRoute);
17  app.use('/auth', authRoute);
18  app.use('/borrow', borrowRoute);
19  app.use('/test', testRoute);
20
21  app.listen(5000, () => {
22    console.log('Server has started on port 5000');
23  });
```

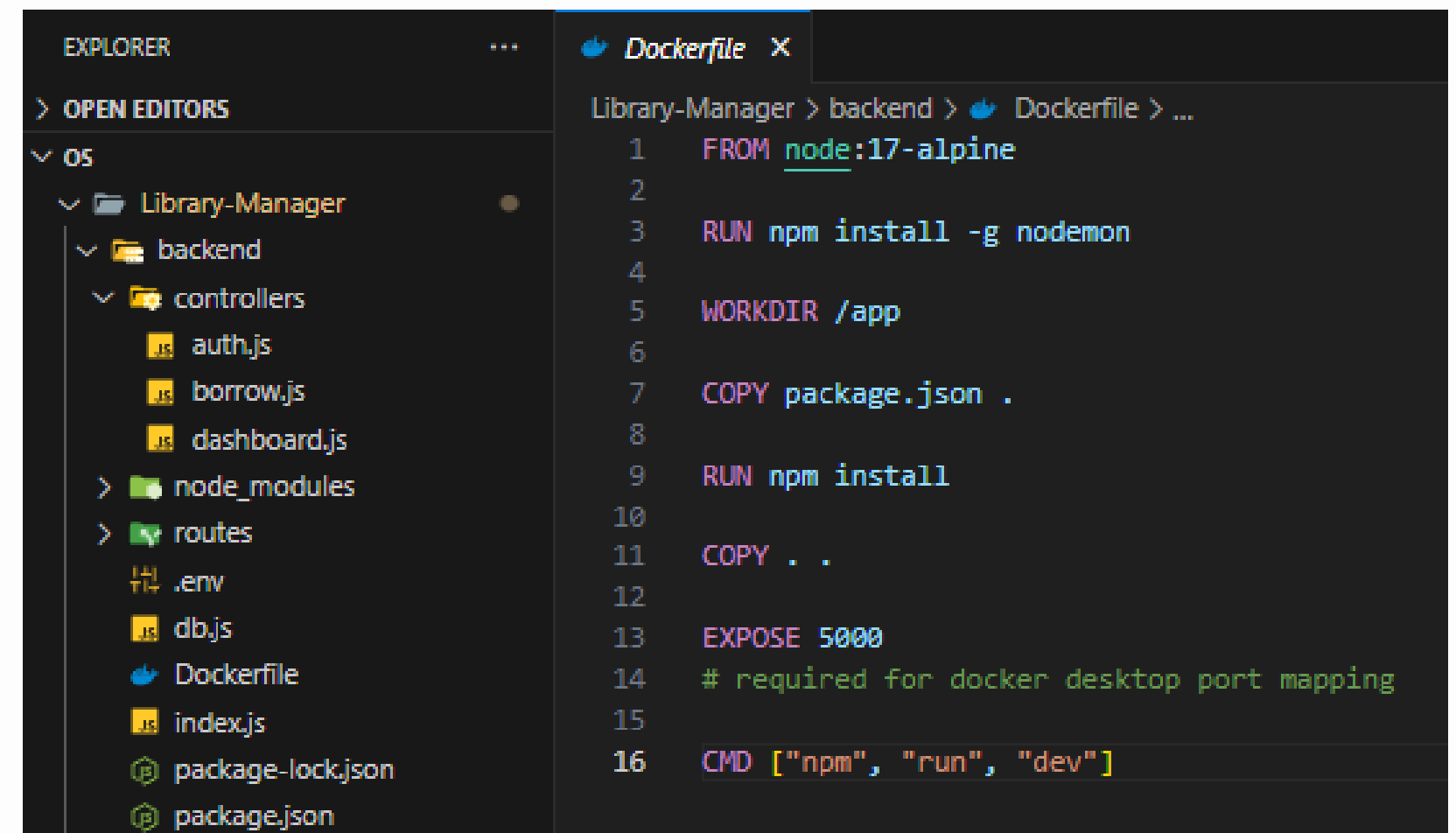
Backend Structure

- controllers/:
 - Contains logic for handling requests:
 - auth.js: Authentication (login, signup).
 - borrow.js: Borrowing logic.
 - dashboard.js: Dashboard data handling.
- routes/:
 - Defines the routes that map to controller functions including, auth.js, borrow.js, dashboard.js, test.js.
- db.js:
 - Manages the connection to the PostgreSQL database.
- .env:
 - Stores environment variables like database credentials.



Backend Dockerfile Structure

- FROM node:17-alpine:
 - Using the Node.js 17 Alpine image ensures that the container is lightweight while supporting the features needed to run the backend service.
- RUN npm install -g nodemon:
 - Installs nodemon globally to allow automatic restarting of the application when changes are detected during development.



The screenshot shows the VS Code interface. On the left, the Explorer view displays a file tree for a project named 'Library-Manager'. The tree structure is as follows:

- Library-Manager
 - backend
 - controllers
 - auth.js
 - borrow.js
 - dashboard.js
 - node_modules
 - routes
 - .env
 - db.js
 - Dockerfile
 - index.js
 - package-lock.json
 - package.json

On the right, the Dockerfile is open in the Editor view. The content of the Dockerfile is as follows:

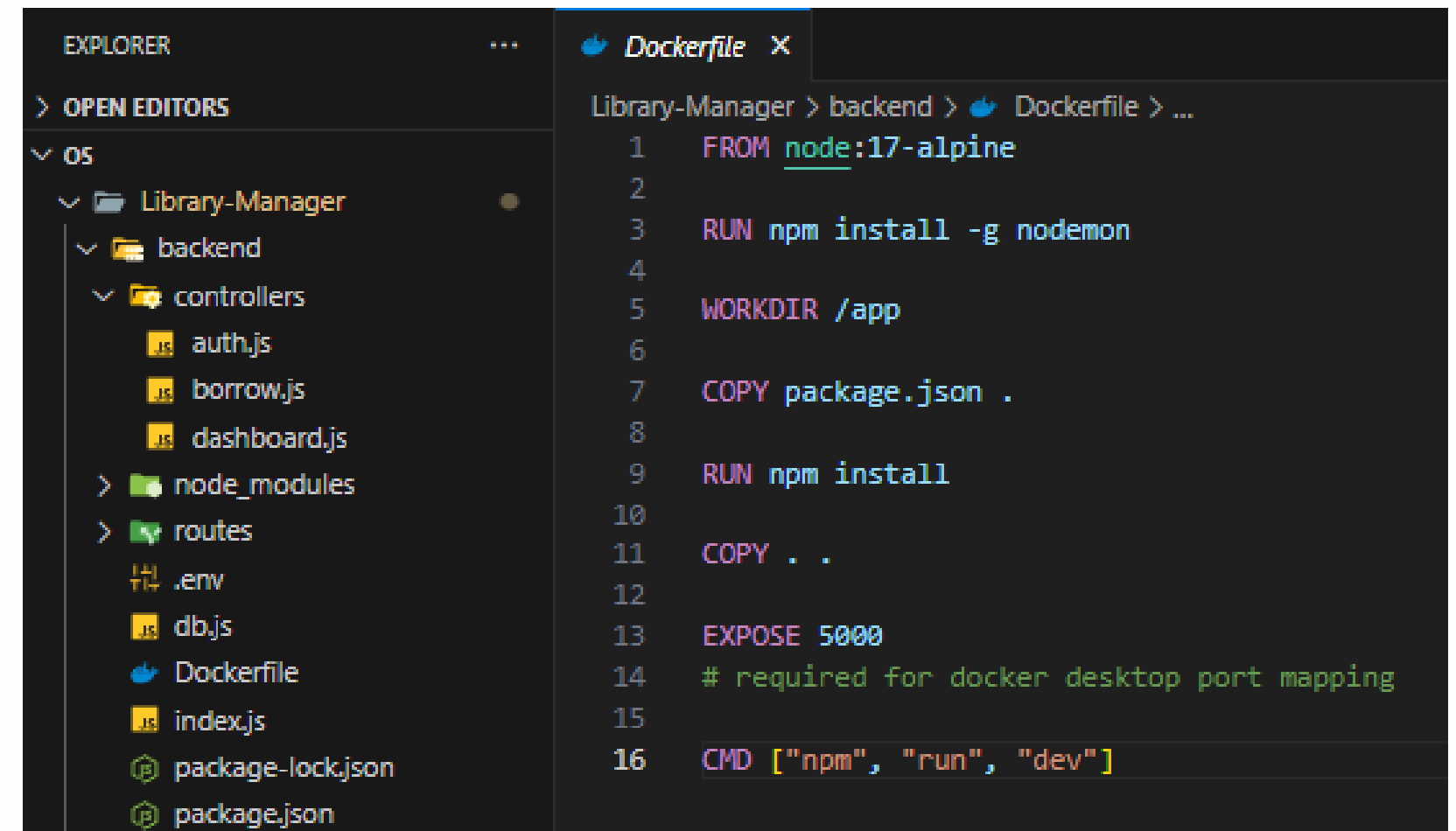
```
1 FROM node:17-alpine
2
3 RUN npm install -g nodemon
4
5 WORKDIR /app
6
7 COPY package.json .
8
9 RUN npm install
10
11 COPY . .
12
13 EXPOSE 5000
14 # required for docker desktop port mapping
15
16 CMD ["npm", "run", "dev"]
```

```
FROM node:17-alpine

RUN npm install -g nodemon
```

Setting Workdir

- WORKDIR /app:
 - Sets /app as the working directory within the container.
 - This ensures that all subsequent Docker commands (copying files, running commands) happen in this directory.
- Why Set Workdir?
 - Ensures consistency, especially when handling multiple files and directories during the build process.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'Library-Manager'. The tree structure is as follows:

- Library-Manager
 - backend
 - controllers
 - auth.js
 - borrow.js
 - dashboard.js
 - node_modules
 - routes
 - .env
 - db.js
 - Dockerfile
 - index.js
 - package-lock.json
 - package.json

On the right, the Dockerfile editor shows the following content:

```
1 FROM node:17-alpine
2
3 RUN npm install -g nodemon
4
5 WORKDIR /app
6
7 COPY package.json .
8
9 RUN npm install
10
11 COPY . .
12
13 EXPOSE 5000
14 # required for docker desktop port mapping
15
16 CMD ["npm", "run", "dev"]
```

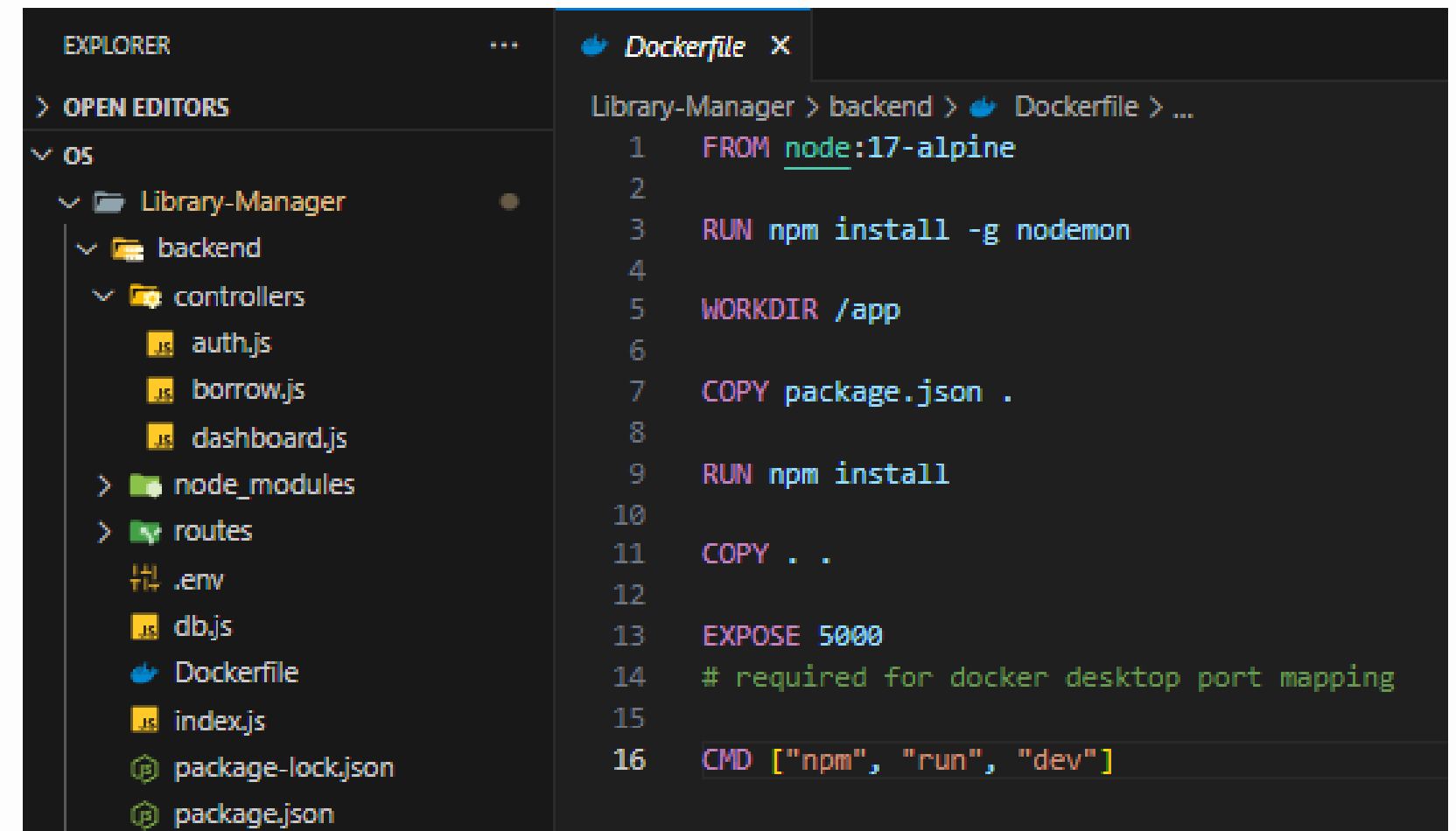


A close-up view of the Dockerfile editor showing line 5:

```
5 WORKDIR /app
```

Installing Dependencies

- COPY package.json:
 - Copies the package.json file into the container to install the dependencies.
- RUN npm install:
 - Installs the dependencies listed in the package.json file (e.g., Express, database libraries, etc.).



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'Library-Manager'. The tree structure is as follows:

- Library-Manager
 - backend
 - controllers
 - auth.js
 - borrow.js
 - dashboard.js
 - node_modules
 - routes
 - .env
 - db.js
 - Dockerfile
 - index.js
 - package-lock.json
 - package.json

On the right, the Dockerfile editor is open, showing the following content:

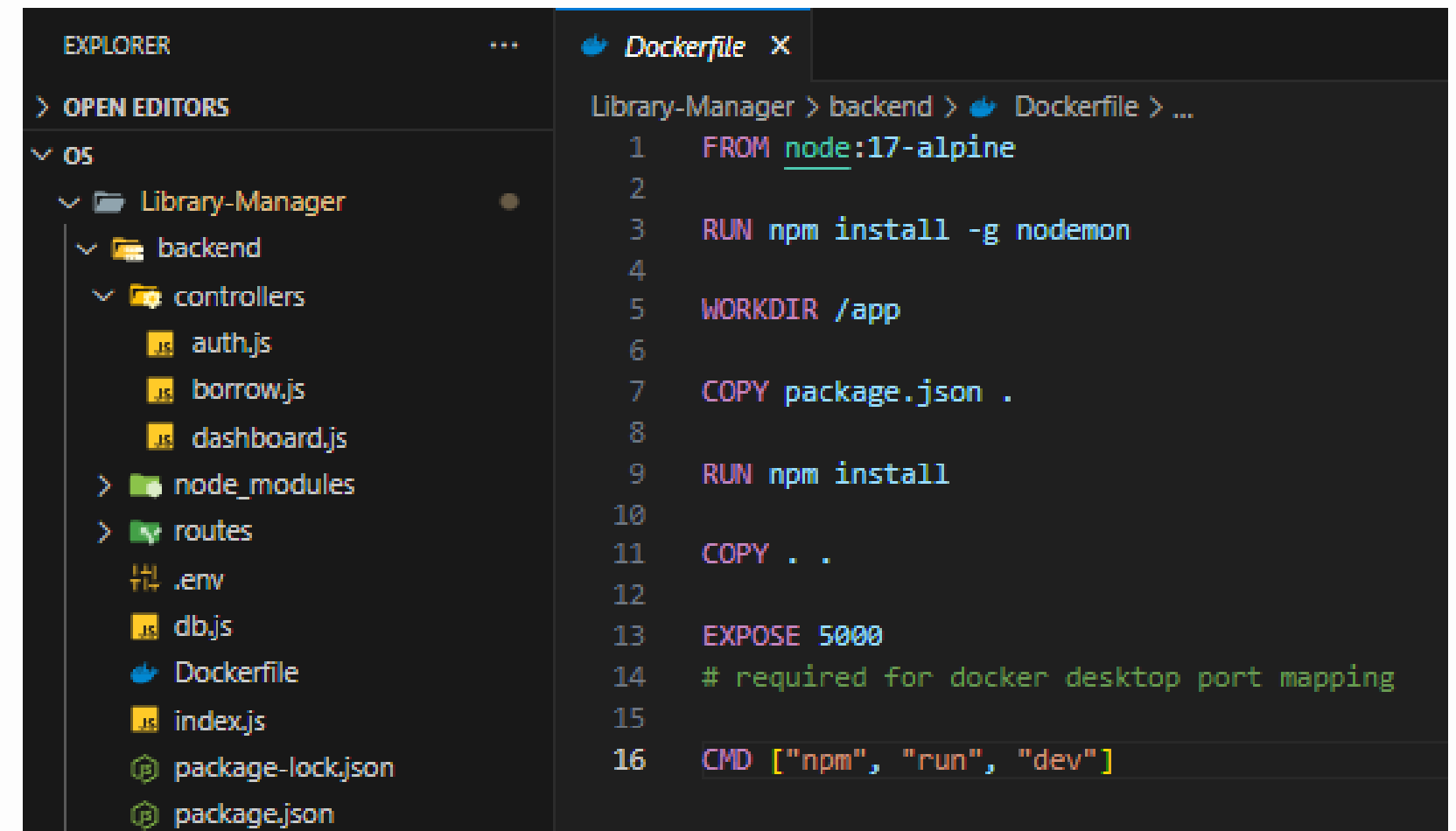
```
1 FROM node:17-alpine
2
3 RUN npm install -g nodemon
4
5 WORKDIR /app
6
7 COPY package.json .
8
9 RUN npm install
10
11 COPY . .
12
13 EXPOSE 5000
14 # required for docker desktop port mapping
15
16 CMD ["npm", "run", "dev"]
```

```
COPY package.json .
```

```
RUN npm install
```


Exposing Ports for Backend

- EXPOSE 5000:
 - Opens port 5000 inside the container, which is where the Express app listens for incoming HTTP requests.
- Why Port 5000?
 - It's a common default port for web servers. The port can then be mapped to the host machine using Docker.



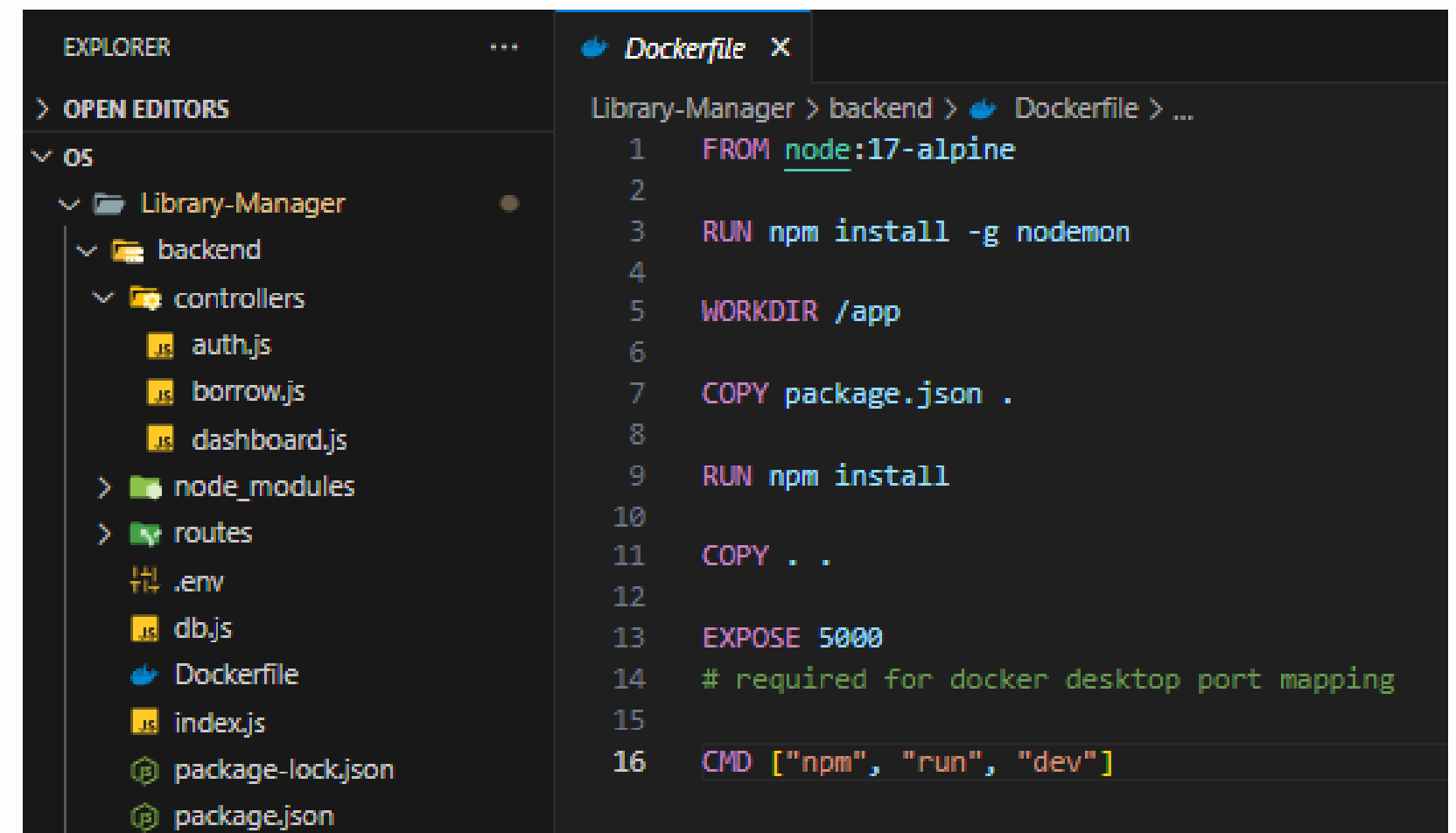
The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays a file tree for a project named 'Library-Manager'. The tree structure is as follows: 'Library-Manager' (folder) contains 'backend' (folder), 'node_modules' (folder), 'routes' (folder), '.env' (file), 'db.js' (file), 'index.js' (file), 'package-lock.json' (file), and 'package.json' (file). The 'backend' folder is expanded, showing 'controllers' (folder) with 'auth.js', 'borrow.js', and 'dashboard.js' (all .js files). On the right, the Dockerfile editor is open, showing the following content:

```
1 FROM node:17-alpine
2
3 RUN npm install -g nodemon
4
5 WORKDIR /app
6
7 COPY package.json .
8
9 RUN npm install
10
11 COPY . .
12
13 EXPOSE 5000
14 # required for docker desktop port mapping
15
16 CMD ["npm", "run", "dev"]
```

```
EXPOSE 5000
# required for docker desktop port mapping
```

CMD

- CMD ["npm", "run", "dev"]:
 - CMD defines the default command that should be executed when the container starts.
 - In this case, it runs the application in development mode using nodemon to auto-reload when files change.



The screenshot shows the Visual Studio Code interface. On the left, the 'EXPLORER' sidebar displays the file structure of a project named 'Library-Manager'. The 'backend' directory is expanded, showing files like 'auth.js', 'borrow.js', 'dashboard.js', 'node_modules', 'routes', '.env', 'db.js', 'Dockerfile', 'index.js', 'package-lock.json', and 'package.json'. On the right, the 'Dockerfile' is open in the editor. The file content is as follows:

```
1 FROM node:17-alpine
2
3 RUN npm install -g nodemon
4
5 WORKDIR /app
6
7 COPY package.json .
8
9 RUN npm install
10
11 COPY . .
12
13 EXPOSE 5000
14 # required for docker desktop port mapping
15
16 CMD ["npm", "run", "dev"]
```

```
EXPOSE 5000
# required for docker desktop port mapping
```

Running Backend Container

Building the Docker Image:

```
PS C:\Users\peera\os\Library-Manager\backend> docker build -t backend .
```

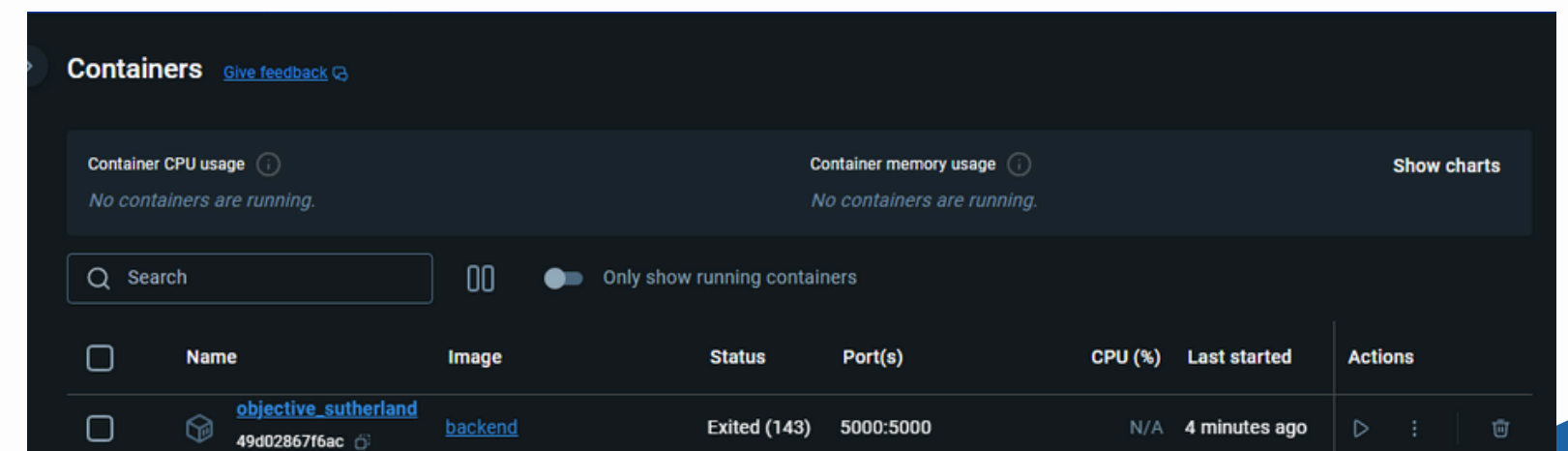
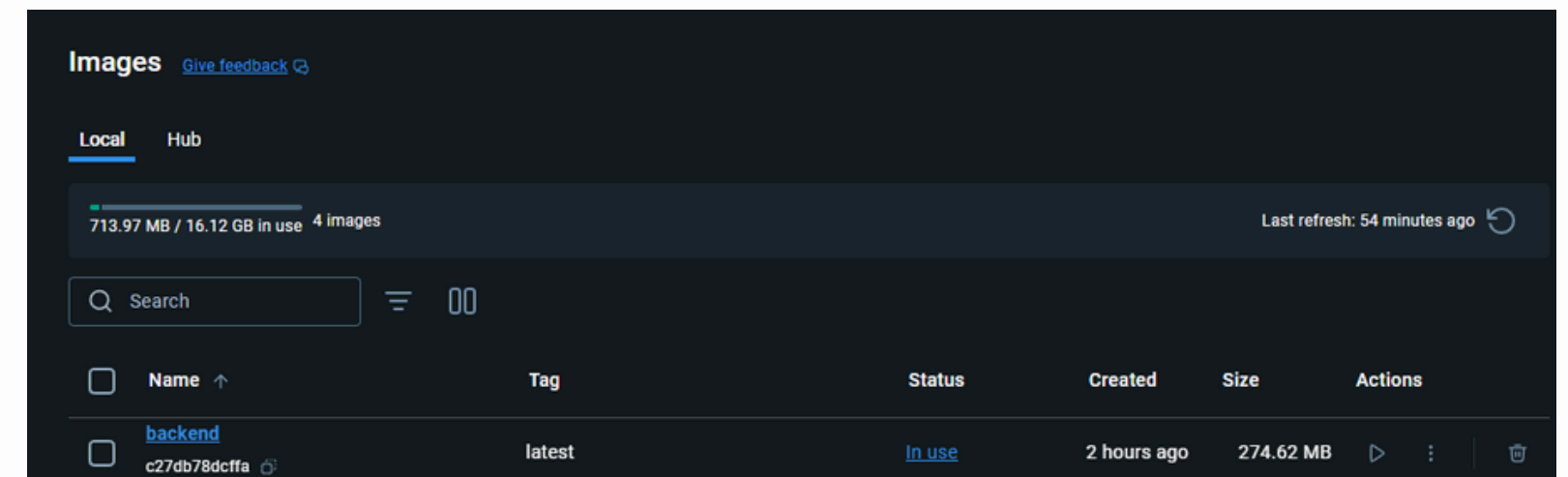
This command creates a Docker image for the backend, tagged as backend.

Running the Container:

```
PS C:\Users\peera\os\Library-Manager\backend> docker run -d -p 5000:5000 backend
```

Runs the container in detached mode (-d), exposing port 5000 on the host machine to map to port 5000 in the container.

Docker Desktop App





DATABASE SERVICE SETUP



Tool Used: Postgres



```
services:
  # PostgreSQL Database Service
  postgres:
    image: postgres:13
    container_name: postgres-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: Menmen553
      POSTGRES_DB: MSC_Library
    ports:
      - "5432:5432" # Expose PostgreSQL on port 5432
    volumes:
      - pgdata:/var/lib/postgresql/data # Persist data between restarts
    networks:
      - app-network # Use the same network for all services
```


Config

- postgres: This is the name of the service.
- image: postgres:13: Specifies the Docker image to use for this service. The image postgres:13 pulls the official PostgreSQL image version 13 from Docker Hub.

```
services:
  # PostgreSQL Database Service
  postgres:
    image: postgres:13
    container_name: postgres-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: Menmen553
      POSTGRES_DB: MSC_Library
    ports:
      - "5432:5432" # Expose PostgreSQL on port 5432
    volumes:
      - pgdata:/var/lib/postgresql/data # Persist data between restarts
    networks:
      - app-network # Use the same network for all services
```

Config

- `container_name`: This sets a specific name for the container running the PostgreSQL database. Instead of Docker generating a random name, this makes it easier to identify and manage the container.

```
services:
  # PostgreSQL Database Service
  postgres:
    image: postgres:13
    container_name: postgres-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: Menmen553
      POSTGRES_DB: MSC_Library
    ports:
      - "5432:5432" # Expose PostgreSQL on port 5432
    volumes:
      - pgdata:/var/lib/postgresql/data # Persist data between restarts
    networks:
      - app-network # Use the same network for all services
```

Config

- environment: This section sets environment variables inside the container, which configure the PostgreSQL instance.
- POSTGRES_USER: Defines the username for accessing the PostgreSQL database. Here, it is set to postgres.
- POSTGRES_PASSWORD: Specifies the password for the above user. In this case, it's set to Menmen553. For security reasons, it's recommended to use a stronger password and consider using a .env file for sensitive information.
- POSTGRES_DB: This variable sets the name of the default database that will be created when the PostgreSQL server starts. Here, it's MSC_Library.

```
services:
  # PostgreSQL Database Service
  postgres:
    image: postgres:13
    container_name: postgres-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: Menmen553
      POSTGRES_DB: MSC_Library
    ports:
      - "5432:5432" # Expose PostgreSQL on port 5432
    volumes:
      - pgdata:/var/lib/postgresql/data # Persist data between restarts
    networks:
      - app-network # Use the same network for all services
```

Config

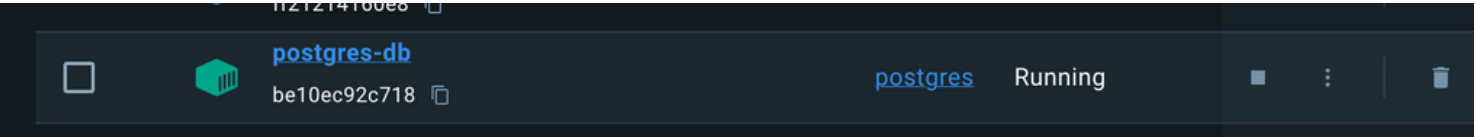
- ports: This maps the container's ports to the host machine's ports.
 - "5432:5432": The first 5432 is the port on your host machine, and the second 5432 is the port in the container where PostgreSQL listens. This configuration allows applications on the host to connect to the PostgreSQL database via port 5432.
- volumes: This section is crucial for data persistence.
 - pgdata: This is the name of the volume created to store PostgreSQL data.
 - /var/lib/postgresql/data: This is the path in the container where PostgreSQL stores its data files. By mounting a volume here, you ensure that data persists even if the container is stopped or removed. When you restart the container, it will have access to the same database data.

```
services:
  # PostgreSQL Database Service
  postgres:
    image: postgres:13
    container_name: postgres-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: Menmen553
      POSTGRES_DB: MSC_Library
    ports:
      - "5432:5432" # Expose PostgreSQL on port 5432
    volumes:
      - pgdata:/var/lib/postgresql/data # Persist data between restarts
    networks:
      - app-network # Use the same network for all services
```

Running the Database Container

Run the docker container for database

DB docker container running



Create the Database through bash in that container

```
postgres=# CREATE DATABASE MSC_Library;
CREATE DATABASE
```

Switch the database

```
Previous connection kept
postgres=# \c msc_library
You are now connected to database "msc_library" as user "postgres"
msc_library=# CREATE TABLE homeworkbooks (
```

```
~/KMITL/3rd year/Web service/Project/Library-Manager feature/docker-feat 02:57:38 AM
> docker run --name postgres-db -e POSTGRES_PASSWORD=Menmen553 -d postgres
be10ec92c718f97fa77799acd2c2de2c278a2ac48425b7ebd51275602c0d3ead

~/KMITL/3rd year/Web service/Project/Library-Manager feature/docker-feat 02:57:43 AM
> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
be10ec92c718   postgres  "docker-entrypoint.s..." 4 seconds ago Up 3 seconds  5432/tcp     postgres-db
```

postgres-db

postgres

be10ec92c718

STATUS

Running (12 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

```
2024-10-11 02:57:43 The files belonging to this database system will be owned by user "postgres".
2024-10-11 02:57:43 This user must also own the server process.
2024-10-11 02:57:43
2024-10-11 02:57:43 The database cluster will be initialized with locale "en_US.utf8".
2024-10-11 02:57:43 The default database encoding has accordingly been set to "UTF8".
2024-10-11 02:57:43 The default text search configuration will be set to "english".
2024-10-11 02:57:43
2024-10-11 02:57:43 Data page checksums are disabled.
2024-10-11 02:57:43
2024-10-11 02:57:43 fixing permissions on existing directory /var/lib/postgresql/data ... ok
2024-10-11 02:57:43 creating subdirectories ... ok
2024-10-11 02:57:43 selecting dynamic shared memory implementation ... posix
2024-10-11 02:57:43 selecting default max_connections ... 100
2024-10-11 02:57:43 selecting default shared_buffers ... 128MB
2024-10-11 02:57:43 selecting default time zone ... Etc/UTC
2024-10-11 02:57:43 creating configuration files ... ok
2024-10-11 02:57:43 running bootstrap script ... ok
2024-10-11 02:57:43 performing post-bootstrap initialization ... ok
2024-10-11 02:57:43 syncing data to disk ... ok
2024-10-11 02:57:43
2024-10-11 02:57:43 Success. You can now start the database server using:
2024-10-11 02:57:43
2024-10-11 02:57:43         pg_ctl -D /var/lib/postgresql/data -l logfile start
2024-10-11 02:57:43
2024-10-11 02:57:43 waiting for server to start....2024-10-10 19:57:43.790 UTC [48] LOG:  starting PostgreSQL 1
5.1 (Debian 16.1-1.pgdg120+1) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 12.2.0-14) 12.2.0, 64-bit
2024-10-11 02:57:43 2024-10-10 19:57:43.791 UTC [48] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGS
```



```

postgres=# \c msc_library
You are now connected to database "msc_library" as user "postgres".
msc_library=# CREATE TABLE borrowedbooks (
    book_id TEXT,
    user_id INTEGER,
    start_date DATE,
    end_date DATE
);
CREATE TABLE
msc_library=# CREATE TABLE muslimahbooks (
    id TEXT,
    category TEXT,
    name TEXT,
    author TEXT,
    amount TEXT,
    room TEXT,
    status TEXT
);
CREATE TABLE
msc_library=# CREATE TABLE muslimeenbooks (
    id TEXT,
    category TEXT,
    name TEXT,
    author TEXT,
    amount TEXT,
    room TEXT,
    status TEXT
);
CREATE TABLE
msc_library=# CREATE TABLE returnedbooks (
    book_id TEXT,
    user_id INTEGER,
    return_date DATE
);
CREATE TABLE
msc_library=# CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255),
    username VARCHAR(100),
    phone_number VARCHAR(20),
    password VARCHAR(100)
);
CREATE TABLE

```

Running the Database Container

View the list of DB

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	IC	
U Rules	Access privileges							
msc_library	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
	=c/postgres							
	postgres=CTc/postgres							
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
	=c/postgres							
	postgres=CTc/postgres							

(4 rows)

View msc_library database schemas

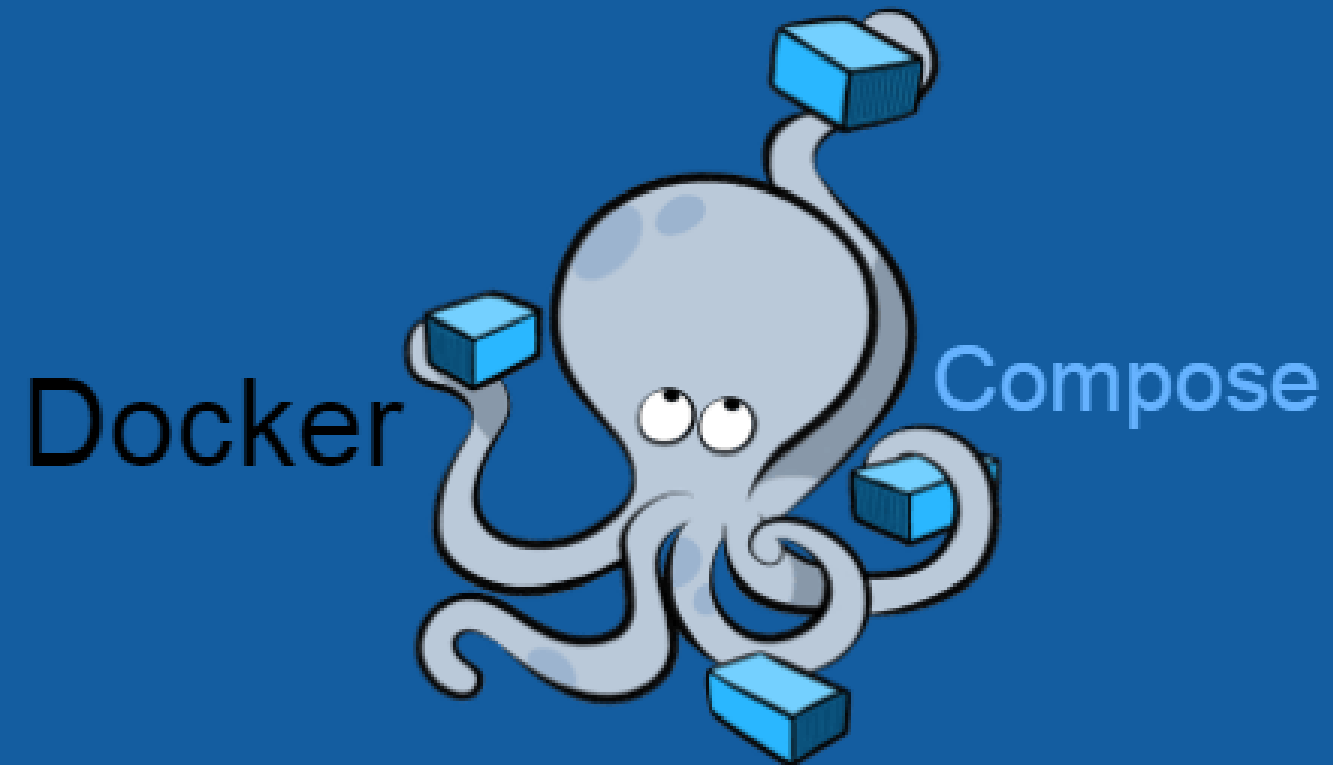
```

CREATE TABLE
msc_library=# \dt

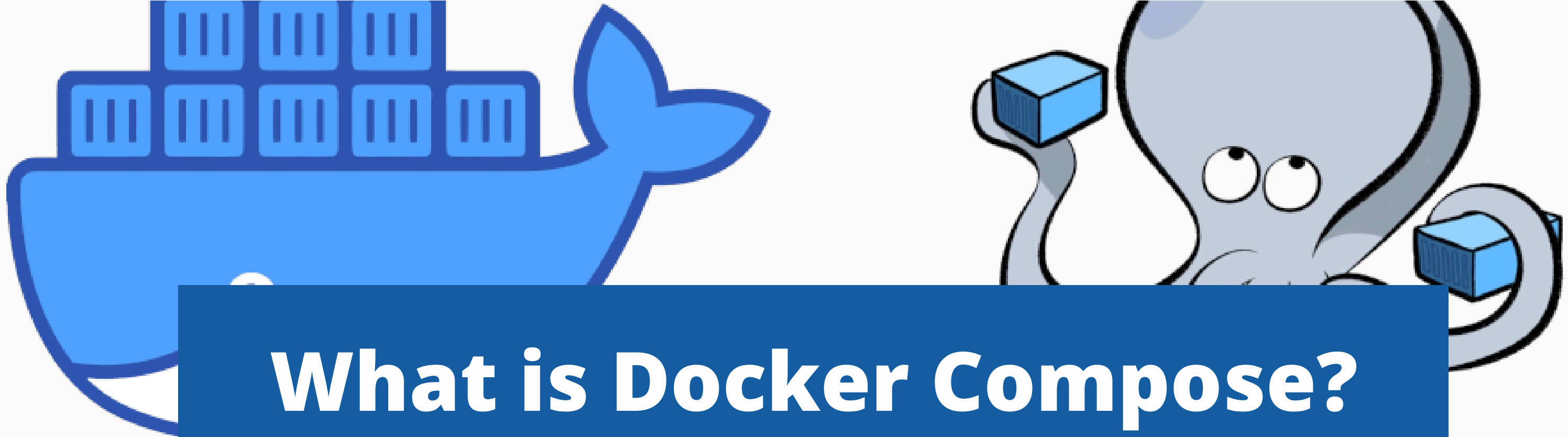
```

List of relations			
Schema	Name	Type	Owner
public	borrowedbooks	table	postgres
public	muslimahbooks	table	postgres
public	muslimeenbooks	table	postgres
public	returnedbooks	table	postgres
public	users	table	postgres

(5 rows)



DOCKER COMPOSE



What is Docker Compose?

It is a tool for defining and managing multi-container Docker applications. It allows you to describe the services your application consists of, along with networks, volumes, and dependencies, in a single YAML file. This simplifies the orchestration and management of multi-service applications.

Docker Compose File Structure

```
EXPLORER
> OPEN EDITORS
  OS
    Library-Manager
      backend
      frontend
      docker-compose.yaml

docker-compose.yaml

1  version: '3.8'
2
3  services:
4    # PostgreSQL Database Service
5    postgres:
6      image: postgres:13
7      container_name: postgres-db
8      environment:
9        POSTGRES_USER: postgres
10       POSTGRES_PASSWORD: Menmen553
11       POSTGRES_DB: MSC_Library
12     ports:
13       - "5432:5432" # Expose PostgreSQL on port 5432
14     volumes:
15       - pgdata:/var/lib/postgresql/data # Persist data between restarts
16     networks:
17       - app-network # Use the same network for all services
18
19   # Backend Node.js Service
20   backend:
21     build:
22       context: ./backend
23     container_name: backend
24     environment:
25       POSTGRES_PASSWORD: Menmen553
26       DB_HOST: postgres # Make sure this matches the Postgres service name
27       DB_PORT: 5432
28       DB_USER: postgres
29       DB_PASSWORD: Menmen553
30       DB_NAME: MSC_Library
31       SESSION_SECRET: hwYnP/ZwhpDox3Xg1Gf7oPibaMI090rB18ncnn0b05I=
32     ports:
33       - "5000:5000" # Expose backend on port 5000
34     depends_on:
35       - postgres # Ensure PostgreSQL starts before backend
36     volumes:
37       - ./backend:/app # Mount the backend directory into the container
38     working_dir: /app
39     command: ["npm", "run", "dev"] # Use your backend start command (e.g., npm run dev or node index.js)
40     networks:
41       - app-network # Use the same network for all services
42
43   # Frontend React Service
44   frontend:
45     build:
46       context: ./frontend # Path to your frontend Dockerfile and code
47       dockerfile: Dockerfile # Specify Dockerfile if not default
48     container_name: frontend
49     depends_on:
50       - backend # Ensure backend starts before frontend
51     ports:
52       - "80:80" # Expose frontend on port 80
53     volumes:
54       - ./frontend:/app # Mount the frontend directory into the container (if needed for development)
55     command: ["nginx", "-g", "daemon off;"] # Nginx command to serve the static files
56     networks:
57       - app-network # Use the same network for all services
58
59   # Define a shared network for services to communicate
60   networks:
61     app-network:
62       driver: bridge
63
64   volumes:
65     pgdata: # Volume to persist PostgreSQL data
```

Services in Docker Compose

PostgreSQL Database Service

```
1  version: '3.8'
2
3  services:
4    # PostgreSQL Database Service
5    postgres:
6      image: postgres:13
7      container_name: postgres-db
8      environment:
9        POSTGRES_USER: postgres
10       POSTGRES_PASSWORD: Menmen553
11       POSTGRES_DB: MSC_Library
12      ports:
13        - "5432:5432" # Expose PostgreSQL on port 5432
14      volumes:
15        - pgdata:/var/lib/postgresql/data # Persist data between restarts
16      networks:
17        - app-network # Use the same network for all services
18
```

```
58
59 # Define a shared network for services to communicate
60 networks:
61   app-network:
62     driver: bridge
63
64 volumes:
65   pgdata: # Volume to persist PostgreSQL data
66
```

Services in Docker Compose

Backend Node.js Service

```
# Backend Node.js Service
backend:
  build:
    context: ./backend
  container_name: backend
  environment:
    POSTGRES_PASSWORD: Menmen553
    DB_HOST: postgres # Make sure this matches the Postgres service name
    DB_PORT: 5432
    DB_USER: postgres
    DB_PASSWORD: Menmen553
    DB_NAME: MSC_Library
    SESSION_SECRET: hwYnP/ZwhpDox3XglGf7oPibaMI090rB18ncnn0b05I=
  ports:
    - "5000:5000" # Expose backend on port 5000
  depends_on:
    - postgres # Ensure PostgreSQL starts before backend
  volumes:
    - ./backend:/app # Mount the backend directory into the container
  working_dir: /app
  command: ["npm", "run", "dev"] # Use your backend start command (e.g., npm run dev or node index.js)
  networks:
    - app-network # Use the same network for all services
```


Services in Docker Compose

Frontend React Service

```
# Frontend React Service
frontend:
  build:
    context: ./frontend # Path to your frontend Dockerfile and code
    dockerfile: Dockerfile # Specify Dockerfile if not default
  container_name: frontend
  depends_on:
    - backend # Ensure backend starts before frontend
  ports:
    - "80:80" # Expose frontend on port 80
  volumes:
    - ./frontend:/app # Mount the frontend directory into the container (if needed for development)
  command: ["nginx", "-g", "daemon off;"] # Nginx command to serve the static files
  networks:
    - app-network # Use the same network for all services
```


Managing Dependencies with depends_on

```
# Backend Node.js Service
backend:
  build:
    context: ./backend
  container_name: backend
  environment:
    POSTGRES_PASSWORD: Menmen553
    DB_HOST: postgres # Make sure this matches the Postgres service name
    DB_PORT: 5432
    DB_USER: postgres
    DB_PASSWORD: Menmen553
    DB_NAME: MSC_Library
    SESSION_SECRET: hwYnP/ZwhpDox3XglGf7oPibaMI090rB18ncnnOb05I=
  ports:
    - "5000:5000" # Expose backend on port 5000
  depends_on:
    - postgres # Ensure PostgreSQL starts before backend
  volumes:
    - ./backend:/app # Mount the backend directory into the container
  working_dir: /app
  command: ["npm", "run", "dev"] # Use your backend start command (e.g.,
  networks:
    - app-network # Use the same network for all services

# Frontend React Service
frontend:
  build:
    context: ./frontend # Path to your frontend Dockerfile and code
    dockerfile: Dockerfile # Specify Dockerfile if not default
  container_name: frontend
  depends_on:
    - backend # Ensure backend starts before frontend
  ports:
    - "80:80" # Expose frontend on port 80
  volumes:
```

- `--build` forces Docker to build images for all the services (frontend, backend, and PostgreSQL) before starting the containers.
- This command starts all services defined in the `docker-compose.yml` file:
 - The PostgreSQL service on port 5432.
 - The backend service (Node.js with Express) on port 5000.
 - The frontend service (React with Nginx) on port 80.



THANK YOU!

Team Members: Peeranat Patarakittisopol 65011446
Tonkla Pokaew 65011610
Stephen